

WEB SISTEMAK

2025/2026 IKASTURTEA

HTTP - HyperText Transfer Protocol

Programazioa. Adibidea Python-ekin (socket liburutegia)
Datu bidalketa: inprimaki formatua. Adibideak.



Web Sistemak by [Oskar Casquero](#) & [María Luz Álvarez](#) is licensed under a [Creative Commons Reconocimiento 4.0 Internacional License](#).

ADIBIDEA PYTHON ERABILIZ: *SOCKET* LIBURUTEGIA

– Python-en *socket* liburutegia erabiliz, ondorengo baliabidea eskatu :

<https://upload.wikimedia.org/wikipedia/commons/7/75/Internet1.jpg>

```
import socket
import ssl

HOST = "upload.wikimedia.org"
PORT = 443

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
context = ssl.create_default_context()
ss = context.wrap_socket(s, server_hostname=HOST)

ss.connect((HOST, PORT))
print("Local socket:", ss.getsockname())

metodo = "GET"
uri = "/wikipedia/commons/7/75/Internet1.jpg"
primera_linea = metodo + " " + uri + " " + "HTTP/1.1\r\n"

cabeceras = { "Host": HOST,
              "User-Agent": "Python Client",
              "Connection": "close" }
cabeceras_cadena = ""
for each in cabeceras:
    cabecera_linea = each + ": " + cabeceras[each] + "\r\n"
    cabeceras_cadena += cabecera_linea

contenido = ""
peticion = primera_linea + cabeceras_cadena + "\r\n" +
contenido
peticion_bytes = bytes(peticion, "utf-8")
```

```
print("\n##### HTTP enviando petición... #####")
print(peticion_bytes)
ss.sendall(peticion_bytes)

respuesta = b""
print("\n##### HTTP recibiendo respuesta... #####")
while True:
    respuesta_trozo = ss.recv(4096)
    if not respuesta_trozo:
        break
    respuesta += respuesta_trozo

print(respuesta)

contenido = respuesta.split(b"\r\n\r\n", 1)[1]
fichero = open("imagen_socket.jpg", "wb")
fichero.write(contenido)
fichero.close()

ss.close()
```

HTTP-REN FUNTZIONAMENDUA: DATU BIDALKETA

- **Zelan bidaltzen da informazioa bezerotik zerbitzarira?**
 - HTTP bezero batek web zerbitzari bati datu mota ezberdinak bidali diezazkioke:
 - datuak formatu binarioan (adibidez: irudiak)
 - datuak JSON edo XML formatuetan
 - **datuak inprimaki formatuan**
- Orain datuen bidalketa inprimaketa formatuan egiten ikasiko dugu, hau da, izen-balio bikote moduan bidaltzen diren datu kate laburrak, alegia.
 - Eredu orokorra: name1=value1&name2=value2&name3=value3
 - Ereduaren adibidea: nan=12345678Z&izena=oskar&abizena=casquero

HTTP-REN FUNTZIONAMENDUA: INPRIMAKIAK -- POST

- Demagun web zerbitzari batean NAN letra kalkulatzen duen baliabide bat dagoela. Baliabide horri *dni* izeneko parametroan NAN zenbakia duen HTTP eskaera bat bidaltzen bazaio, baliabideak NAN letra duen HTTP erantzun bat itzultzen du.
- Datua inprimaki formatuan bidaltzeko, bezeroak HTTP eskaera horrela definitzen du:
 - Metodoa: POST
 - Mezuaren gorputza bereizteko goiburuak:
 - Content-Type: application/x-www-form-urlencoded
 - Content-Length
 - Izen-balio bikoteak era jakin batean kodifikatu (UTF-8 + ehuneko kodifikazioa) eta formateatu (izena=balioa eta "&" bikoteen arteko bereizle bezala) egin behar dira

Eskaera baten adibidea

```
POST /resource HTTP/1.1
Host: ws2025.com
Accept: text/plain
Content-Type: application/x-www-form-urlencoded
Content-Length: 12
User-Agent: Python bezeroa

dni=12345678
```

Erantzun baten adibidea

```
HTTP/1.1 200 OK
Date: Wed, 25 Nov 2015 08:07:43 GMT
Content-Length: 1
Content-Type: text/plain

Z
```

HTTP-REN FUNTZIONAMENDUA: INPRIMAKIAK -- GET

- Izen-Balio bikoteak mezuaren gorputzean bidali beharrean, URI-an bertan bidali daitezke “?” karakterea juntagailu bezela erabiliz → *query string*
 - Metodoa: GET
 - Ez dira *Content-Type* eta *Content-Length* goiburuak erabili behar
 - Query String-ean inprimaki formatudun datuak bakarrik bidali daitezke.
 - HTTP eskaeraren egiturak query string-a mugatzen du: izen-balio bikoteen katea “?” karakterearen eta “ HTTP/1.1” katearen tartean kokatzen da.
 - Datuen kodifikazioa eta formatua lehengo moduan egiten da.
 - Desabantaila: bidali daitekeen datu koputua mugaturik dago: 1024 zortzikote.

Eskaera baten adibidea

```
GET /recurso?dni=12345678 HTTP/1.1
Host: ws2025.com
Accept: text/plain
User-Agent: Python bezeroa
```

Erantzun baten adibidea

```
HTTP/1.1 200 OK
Date: Wed, 25 Nov 2015 08:07:43 GMT
Content-Length: 1
Content-Type: text/plain
```

Z

HTTP-REN FUNTZIONAMENDUA: INPRIMAKIAK -- LABURPENA

GET /app/servlet/contactUs?name=XXX&email=YYY&message=ZZZ HTTP/1.1

Host: ws2025.com

Accept: text/html

Accept-Language: en-US,en

Connection: keep-alive

POST /app/servlet/contactUs HTTP/1.1

Host: ws2025.com

Accept: text/html

Accept-Language: en-US,en

Connection: keep-alive

Content-Type: application/x-www-form-urlencoded

Content-Length: 30

name=XXX&email=YYY&message=ZZZ

HTTP-REN FUNTZIONAMENDUA: INPRIMAKIAK (HTML)



```
<html>
  <head>
    <title>Contact Us</title>
  </head>
  <body>
    <p>
      <form method="GET" action="/app/servlet/contactUs">
        <b>Izena: </b>
        <input type="text" name="name">
        <br/>
        <b>Posta elektronikoa: </b>
        <input type="text" name="email">
        <br/>
        <b>Mezua: </b>
        <input type="text" name="message">
        <br/><br/>
        <input type="submit">
      </form>
    </p>
  </body>
</html>
```

(*) image extracted from <http://www.freedback.com/>

HTTP-REN FUNTZIONAMENDUA:

DATUEN KODIFIKAZIOA INPRIMAKI FORMATUAN

- Inprimaki formatuan, edukiarentzako ondorengo kodifikazio prozedura zehazten da izen-balio bikotearen izen eta balio kateentzat:
 - Karaktere berezi bat (URI-an edo izen-balio bikote katean bereizle diren karaktereak) edo [US-ASCII](#) alfabetoan agertzen ez den karaktere bat kodifikatu nahi bada (adibidez, azentua duten karaktereak), izen-balio bikote katea osatu aurretik bi pausu burutu behar dira:
 1. Karakterea [UTF-8](#)-n kodifikatu.
 - Adibidea 1: & → 26
 - Adibidea 2: á → C3 A1
 2. Aurreko pausuaren emaitza “ehuneko” kodifikazioan (percent encoding)
 - Adibidea 1: 26 → %26
 - Adibidea 2: C3 A1 → %C3 %A1

OHARRA: goiko adibide bakoitzean, karaktere bakarra kodifikatzen da; baina kodifikazio prozeduraren eraginez, zenbat karaktere bidaliko dira?

- Adibidea 1: %26 → 3 karaktere: %, 2, 6
- Adibidea 2: %C3 %A1 → 6 karaktere: %, C, 3, %, A, 1

HTTP-REN FUNTZIONAMENDUA:

DATUEN KODIFIKAZIOA INPRIMAKI FORMATUAN

- Adibidea:
 - bidali nahi diren datuak: izena=Iñaki y abizena=Pérez
 - mezuaren gorputzean kodifikatuko diren datuak:
izena=I%C3%B1aki&abizena=P%C3%A9rez
- izen-balio bikotearen izen eta balio kateetan zuriune bat agertzen bada, hau “+” batekin kodifikatzen da:
 - bidali nahi diren datuak: izena_abizena=Iñaki Pérez
 - mezuaren gorputzean kodifikatuko diren datuak:
izena_abizena=I%C3%B1aki+P%C3%A9rez
- “+” izen-balio bikotearen izen eta balio kateetan datu bezela erabili nahi bada, orduan UTF-8 + % kodifikazio prozedura jarraitu behar da:
 - bidali nahi diren datuak: num1=2, num2=2 y operator=+
 - mezuaren gorputzean kodifikatuko diren datuak:
num1=2&num2=2&operator=%2B

ADIBIDEA:

DATU BIDALKETA INPRIMAKI FORMATUAN -- POST

- NAN letraren zenbakia lortu: <http://ws-sendingformdata.appspot.com/>

Bezeroaren eskaera

POST /processForm HTTP/1.1
Host: ws-sendingformdata.appspot.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 12

nan=12345678

Zerbitzariaren erantzuna

HTTP/1.1 200 OK
Date: Wed, 25 Nov 2015 08:07:43 GMT
Server: Google Frontend
Content-type: text/html; charset=utf-8
Content-Length: 1

Z

Datuen
kodifikazioa

```
import requests
import urllib
import sys
```

```
metodoa = 'POST'
uria = "http://ws-sendingformdata.appspot.com/processForm"
goiburuak = {'Host': 'ws-sendingformdata.appspot.com',
             'Content-Type': 'application/x-www-form-urlencoded'}
edukia = {'nan': sys.argv[1]}
edukia_encoded = urllib.parse.urlencode(edukia)
goiburuak['Content-Length'] = str(len(edukia_encoded))
erantzuna = requests.request(metodoa, uria, data=edukia_encoded,
                             headers=goiburuak, allow_redirects=False)
```

```
kodea = erantzuna.status_code
deskribapena = erantzuna.reason
print(str(kodea) + " " + deskribapena)
for goiburua in erantzuna.headers:
    print(goiburua + ": " + erantzuna.headers[goiburua])
edukia = erantzuna.content
print(edukia)
```

ADIBIDEA:

DATU BIDALKETA INPRIMAKI FORMATUAN -- GET

- NAN letraren zenbakia lortu: <http://ws-sendingformdata.appspot.com/>

Bezeroaren eskaera

GET /processForm?nan=12345678 HTTP/1.1
Host: ws-sendingformdata.appspot.com

Zerbitzariaren erantzuna

HTTP/1.1 200 OK
Date: Wed, 25 Nov 2015 08:07:43 GMT
Server: Google Frontend
Content-type: text/html; charset=utf-8
Content-Length: 1

Z

```
import requests  
import urllib  
import sys
```

```
metodoa = 'GET'  
base_urria = "http://ws-sendingformdata.appspot.com/processForm"  
goiburuak = {'Host': 'ws-sendingformdata.appspot.com'}  
edukia = {'nan': sys.argv[1]}  
edukia_encoded = urllib.parse.urlencode(edukia)  
uria= base_urria + '?' + edukia_encoded  
erantzuna = requests.request(metodoa, uria,  
                             headers=goiburuak, allow_redirects=False)
```

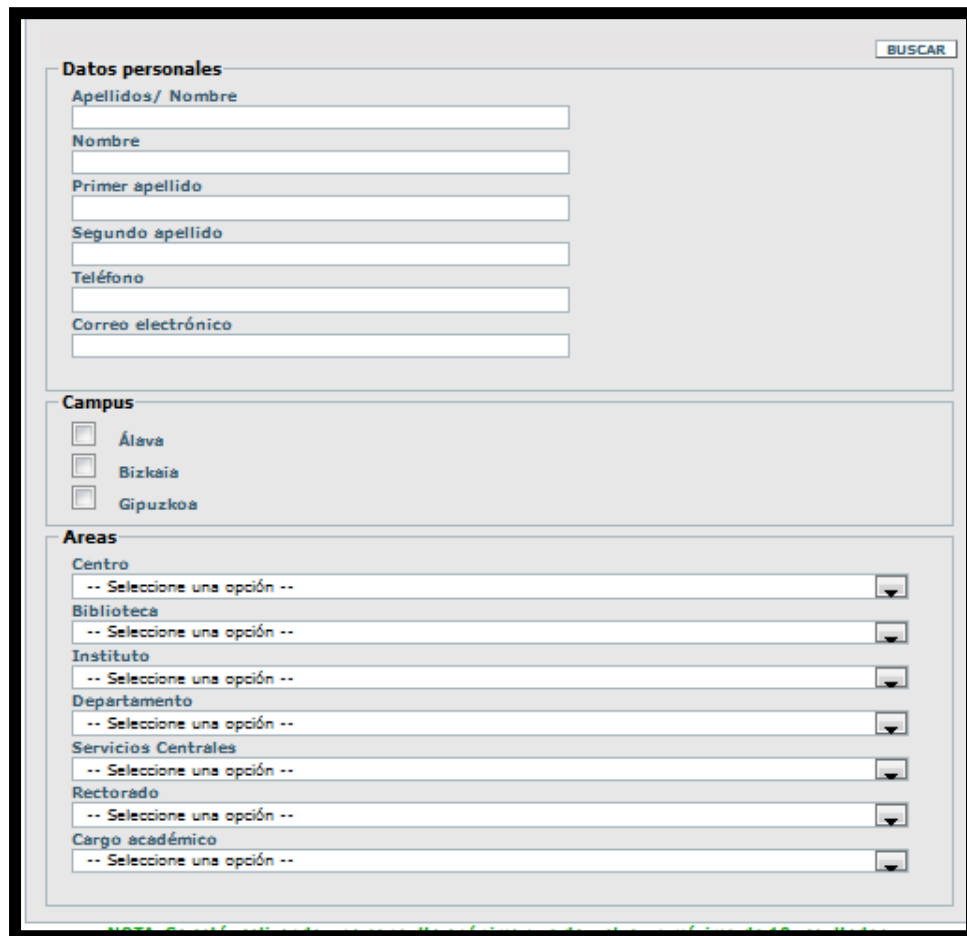
```
kodea = erantzuna.status_code  
deskribapena = erantzuna.reason  
print(str(kodea) + " " + deskribapena)  
for goiburua in erantzuna.headers:  
    print(goiburua + ": " + erantzuna.headers[goiburua])  
edukia = erantzuna.content  
print(edukia)
```

Datuak
URLan

ARIKETA: KONTSULTA EZAZU EHUko DIREKTORIOA

BURP ETA PYTHON ERABILIZ

- <https://www.ehu.eus/bilatu/buscar/bilatu.php?lang=es>



The screenshot shows a web form titled "Datos personales" with a "BUSCAR" button. It contains several input fields for personal information: Apellidos/ Nombre, Nombre, Primer apellido, Segundo apellido, Teléfono, and Correo electrónico. Below this is a "Campus" section with three checkboxes for Álava, Bizkaia, and Gipuzkoa. The "Areas" section contains several dropdown menus for selection: Centro, Biblioteca, Instituto, Departamento, Servicios Centrales, Rectorado, and Cargo académico. Each dropdown menu has a placeholder text "-- Seleccione una opción --".

POST /bilatu/buscar/sbilatu.php HTTP/1.1
Host: www.ehu.eus
Content-Type: application/x-www-form-urlencoded
Content-Length: 16

abi_ize=armentia